Lab 03 - Compiler Construction

Outcome of this lab

At the end of this lab, you will be able to:

- explain what a lexical analyzer does and distinguish tokens from lexemes;
- describe the three sections of a Flex program and its lexical rules;
- generate a scanner using Flex and GCC, and identify tokens and lexical errors in C code.

Lexical Analysis Using Flex

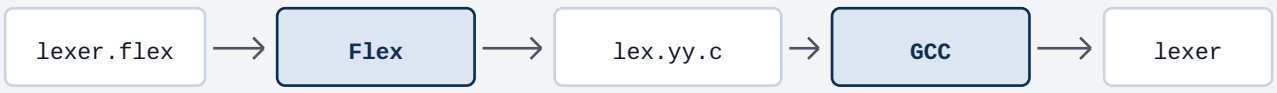
Lexical analysis is the first phase of a compiler. It reads source code and groups characters into meaningful units called **tokens**. The actual characters matched are called **lexemes**.

```
int age = 10;
```

For this line, **int** is a keyword, **age** an identifier, **=** an operator, **10** a number, and **;** a separator.

How Flex works

Flex turns the patterns and actions in `lexer.flex` into a C scanner. GCC then compiles that generated C file into a program that can analyze `source.c`.



Run the generated scanner with source.c as input

```
./lexer < source.c > tokens.txt
```

Files for this lab

- **lexer.flex** - the supplied Flex program defining token patterns and actions.
- **source.c** - the supplied sample C program to scan.

Before beginning, make sure Flex and GCC are installed as described in Lab 01.

The Sample Program: source.c

The supplied `source.c` is the input program for this lab. Read it carefully before running the lexical analyzer.

```
int main() {
  int @ = 10;
  // This is single line comment
  10 = x;
  return 0;
}
```

Generating and Running the Scanner

Keep `lexer.flex` and `source.c` in the same folder. Open a terminal in that folder.

Linux (Ubuntu/Debian)

• Generate the scanner:

```
flex lexer.flex
```

• Compile the generated C file:

```
gcc lex.yy.c -o lexer
```

• Run and save the output:

```
./lexer < source.c > tokens.txt
```

Windows (MSYS2 UCRT64)

Open the **MSYS2 UCRT64** terminal, as used in Lab 01.

```
flex lexer.flex
gcc lex.yy.c -o lexer.exe
./lexer.exe < source.c > tokens.txt
```

Mac (Apple Silicon)

Use the Flex and GCC commands configured in Lab 01.

```
flex lexer.flex
gcc lex.yy.c -o lexer
./lexer < source.c > tokens.txt
```

View the result

On Linux, MSYS2, or Mac, read the output file using:

```
cat tokens.txt
```

The < symbol feeds `source.c` to the lexer. The > symbol writes the printed output into `tokens.txt`.

Do the following tasks

1. Study the provided `lexer.flex` and explain its configuration, regular expressions, lexical rules, and C functions in your own words.
2. Run the lexer on `source.c`. Include the contents of `tokens.txt` in your report and explain the recognized tokens, ignored text, and any reported error.

Scope of this lexer

This is a teaching scanner, not a full C lexer. It recognizes a small set of tokens and single-line comments. It does not completely handle floating-point literals, string literals, or multi-line comments.